

ASSIGNMENT 3 — BRIDGE PATTERN

Student: Yernur Alimbekov

Group: SE-2524

Course: Software Design Patterns

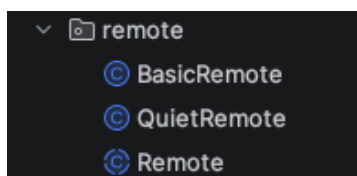
Topic: D — Remote Controls

1. INTRODUCTION

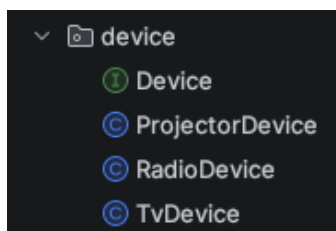
This project demonstrates the Bridge Design Pattern using remote controls and devices.

The application contains two independently varying dimensions.

The first dimension is the abstraction hierarchy:



The second dimension is the implementation hierarchy:



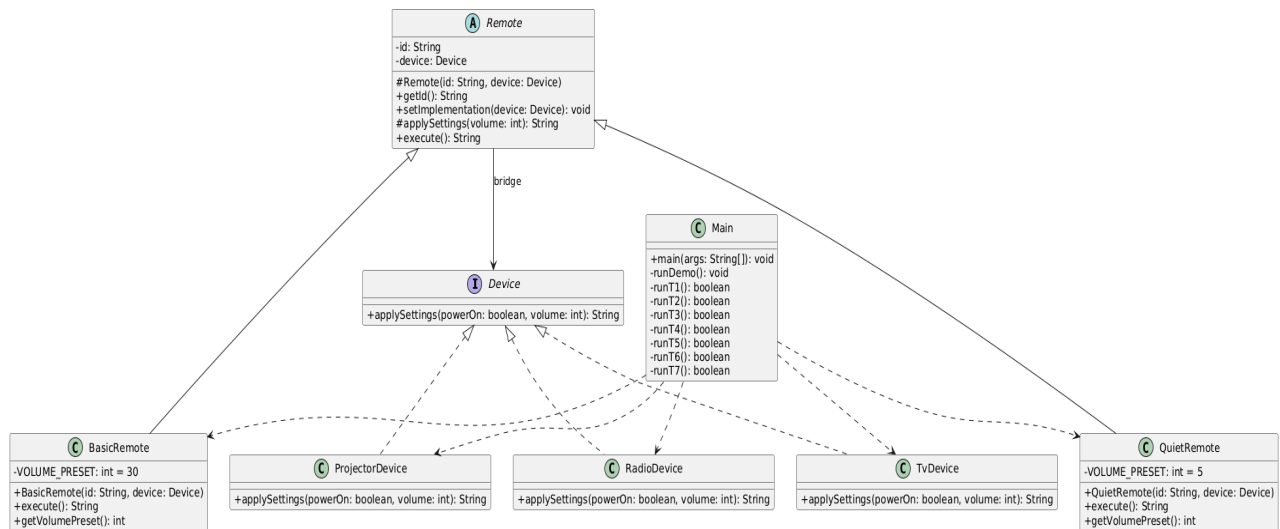
The Remote abstraction stores a reference of type Device. Therefore, a remote control does not depend directly on a concrete device implementation.

Bridge is suitable for this application because remote-control behavior and device implementation can change independently.

For example, BasicRemote always applies its normal volume preset of 30, while QuietRemote applies a quiet volume preset of 5. At the same time, TvDevice, RadioDevice, and ProjectorDevice determine how the device-specific result is produced.

Without Bridge, the program could require separate classes for every combination, such as BasicTvRemote, BasicRadioRemote, QuietTvRemote, and QuietRadioRemote. Bridge avoids this class explosion by connecting the two hierarchies through composition.

2. UML CLASS DIAGRAM



The UML diagram shows the two separate hierarchies.

Remote is the abstraction and contains an interface-typed Device reference.

BasicRemote and QuietRemote are refined abstractions.

Device is the Implementor interface. TvDevice, RadioDevice, and ProjectorDevice are concrete implementations of this interface.

Main acts as the client and creates different combinations of Remote and Device implementations.

The connection from Remote to Device represents the bridge between the two hierarchies.

3. BRIDGE IMPLEMENTATION

The central part of the Bridge implementation is the Device reference stored inside Remote:

```
private Device device;
```

The concrete implementation is supplied through the constructor:

```
protected Remote(String id, Device device) {
    this.id = Objects.requireNonNull(id, message: "ID cannot be null");
    this.device = Objects.requireNonNull(device, message: "Device cannot be null");
}
```

Remote depends only on the Device interface. It does not construct TvDevice, RadioDevice, or ProjectorDevice and does not check their concrete types.

The implementation can also be replaced while the Remote object continues to exist:

```
public void setImplementation(Device device) {  
    this.device = Objects.requireNonNull(device, message: "Device cannot be null;");  
}
```

This behavior is demonstrated by test T5. The same BasicRemote object initially uses TvDevice and is later switched to RadioDevice.

Reference equality is checked using:

```
originalReference = remote;
```

The result is true, which proves that the abstraction object itself was not replaced.

The ID and volume preset are also checked before and after the implementation replacement. They remain unchanged, while the implementation-specific result changes from TV to RADIO.

4. CLEAN CODE PRACTICES

4.1 Separation of Responsibilities

The Device interface contains the low-level device operation:

```
public interface Device { 3 implementations  
    String applySettings(boolean powerOn, int volume);  
}
```

Device implementations are responsible for device-specific output, while Remote classes are responsible for remote-control behavior.

Benefit:

Each class has a clear responsibility. Remote classes do not contain device-specific logic, and device classes do not contain remote-control behavior.

4.2 Meaningful Names

Examples of meaningful names used in the project include:

BasicRemote

QuietRemote

ProjectorDevice

applySettings()

```
setImplementation()
```

```
getVolumePreset()
```

Benefit:

The names describe the role and purpose of each class and method. This makes the program easier to understand without requiring unnecessary comments.

4.3 Small and Focused Methods

BasicRemote contains a small execute() method:

```
@Override
public String execute() {
    return applySettings(VOLUME_PRESET);
}
```

The method has one responsibility: execute BasicRemote behavior using its volume preset.

Benefit:

Small methods are easier to understand, test, and modify.

4.4 Avoidance of Duplication

The common delegation workflow is placed in Remote:

```
protected String applySettings(int volume) {
    return device.applySettings(powerOn: true, volume);
}
```

BasicRemote and QuietRemote reuse this method instead of duplicating the complete device-control logic.

Benefit:

Common behavior exists in one location. Changes to the delegation workflow do not need to be repeated in multiple refined abstractions.

4.5 Encapsulation of State

Remote keeps its ID and implementation reference encapsulated:

```
private final String id;
```

```
private Device device;
```

The ID cannot be changed after construction, while the Device reference can only be replaced through setImplementation().

Benefit:

The internal state is controlled by the class. This protects the abstraction state while still allowing the implementation side of the Bridge to change at runtime.

5. INDEPENDENT EXTENSION

The initial working implementation contained:

- Remote
- BasicRemote
- QuietRemote
- Device
- TvDevice
- RadioDevice
- Main tests T1-T5

This version was saved in the base commit:

093b233f24d7ad7cd7027e111a0fa05b15e85408

After the base version was completed, ProjectorDevice was added as the third implementation.

ProjectorDevice implements the existing Device interface:

```
package bridge.device;

public class ProjectorDevice implements Device{
    @Override
    public String applySettings(boolean powerOn, int volume) {
        String powerState = powerOn ? "ON" : "OFF";

        return "PROJECTOR | power=" + powerState + " | volume=" + volume;
    }
}
```

The existing Remote hierarchy, Device interface, TvDevice, and RadioDevice did not need to be changed.

Only ProjectorDevice was added and Main was updated to demonstrate T6 and T7.

This is evidence that the implementation hierarchy can be extended independently from the abstraction hierarchy.

The exact source changes are recorded in extension.diff.

6. BRIDGE VS ADAPTER

Bridge and Adapter may both use composition, but their intent is different.

Bridge is used when two dimensions of a system are intentionally designed to vary independently. In this project, remote-control behavior and device implementations are separated from the beginning.

Adapter is normally used when two existing interfaces are incompatible. An Adapter converts the interface of an existing class into an interface that the client expects.

Therefore:

Bridge separates abstractions and implementations by design.

Adapter makes incompatible existing components work together.

7. TRADE-OFF

One trade-off of the Bridge Pattern is increased structural complexity.

Instead of using one simple hierarchy, the application contains separate abstraction and implementation hierarchies connected through an interface. This introduces more classes and relationships.

For a very small application this may be unnecessary. However, when both dimensions need to grow independently, the additional structure improves extensibility and prevents a large number of combination subclasses.

8. CONCLUSION

The project successfully demonstrates the Bridge Pattern using remote controls and devices.

All required combinations are demonstrated by T1-T4. T5 proves runtime implementation switching on the same abstraction object while preserving its ID and domain state.

ProjectorDevice was added after the base implementation without modifying the existing hierarchies, and T6-T7 demonstrate the new implementation.

The final demo produces:

SUMMARY: 7/7 PASS

REFERENCES

1. Lecture 4 — Bridge Pattern, Software Design Patterns course materials.
2. Assignment 3 — Bridge Pattern Instructions, Astana IT University.